SDA [2]

first step of developing software – clean code

General Rules ⇒ 1. General Conventions

applicable for one, design & anything

varies from company to company

getUsers – get_user

2. Keep it simple, stupid

```
if user=true          | if (user)
print (welcome)       |   print (  )
```

→ Better

3. Boy Scout Rule

make small improvement on refactoring over time. (keep regularity)

4. Find the root cause

(dont do quick fixes – even happens in industries cause of deadline)

Design Rules

1. Configurable Files

T, str etc

```
        a   b   c   d   e  → files
db.connect()  –   –   –   –   –
```

↓ duplication, security issues

use 1 config file / global data.

2. Prefer polymorphism, Inheritence

```
if (shape = ...)
else if (shape = rect)
```
×

```
class shape
```
✓

3. Multi Threading files | For refactoring, bug fix, small files are better |

4. Over Configuration Prevention

using this values once

logistic Reg -> lots of parameters (set some default values of some parameters)

only the imp. ones to so that when we call the func we dont need to provide that much param.

5. Dependency ~~Inversion~~ Injection

```
Class Service()
def __init__ (self, db)
  self.db = db
```
×

```
def __init__ (self)
self.db =
  Database();
```
✓

tight coupling

6. follow the rules of Demeter

person.user.student.dept.program ✗

dept.program ←

Understandibility tips

Proper Documentation
Better ~~Ma~~ Naming (based on context)

1. Be Consistent
2. Better naming convention
3. Handle edge cases

valid (user) ?
api call
valid (age)

] keep things in chunks

4. if (!no user) ✗ double neg.

...

if (user) ←

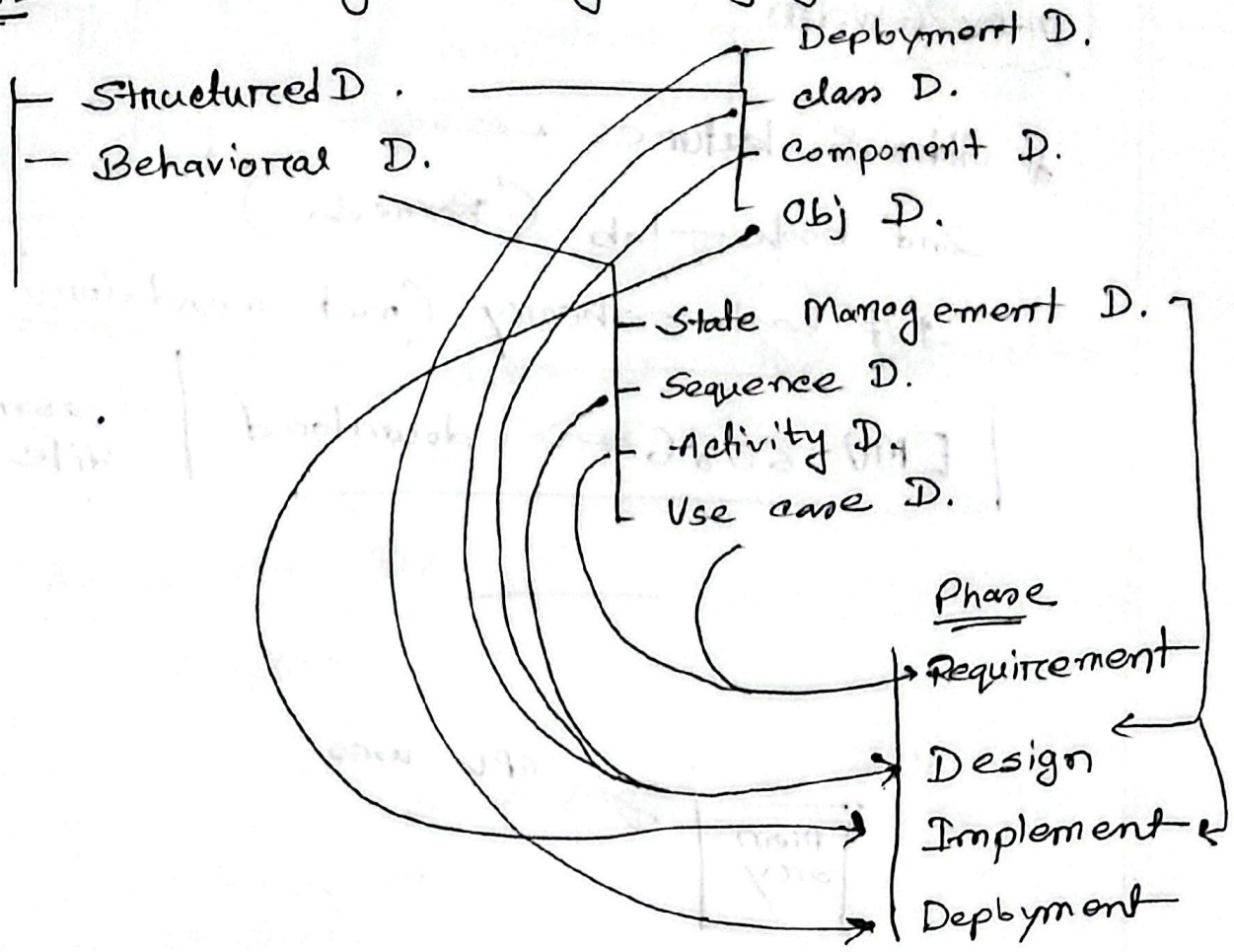
...

nested cond. ✗ → over complicates

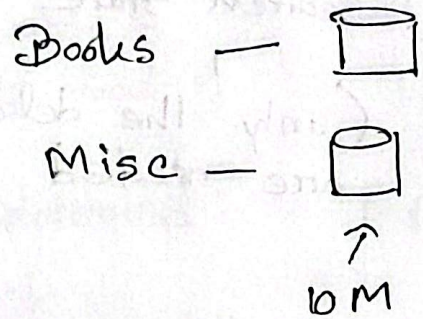
5. function Rules

single resp, less arg, param. proper name, small in length

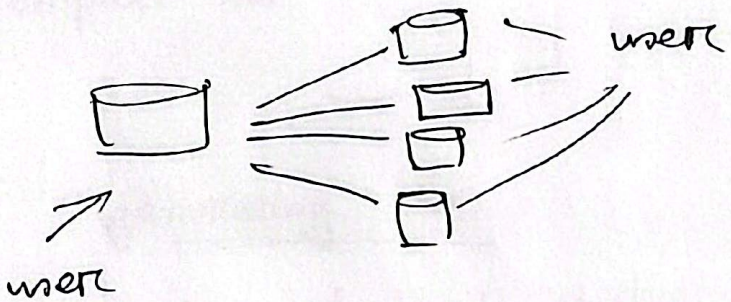
UML - Unifying Modeling Language



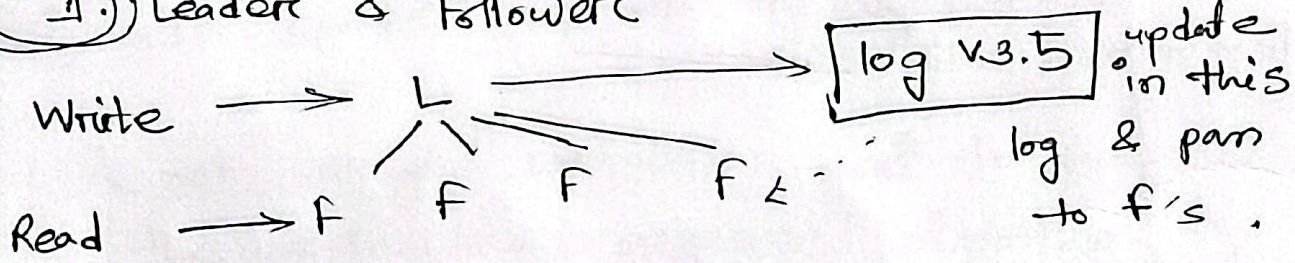
Sharding



Database Replication



1. Leader & Followers



but how ?
& when ? successful.

if updates has reach all the f_then succesfut write.

write
success

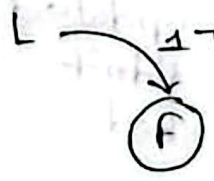
Synchronous Replication

Asynchronous

* How can we decide how many followers?

→ initially we'll take a few f, then increase as per need.

steps → copy specific data from L to the new F.



prob - Duplication

Soln → snapshot (by maintaining log version)

{ postgre - log sequence no.
MySQL - bin log coordinates

// what happens when f crashes

// " " when L " [Temporarily promote another f to L]

but still some data may loose

- by voting assigned from the start
- synchronous replicated f will be L
- most updated f

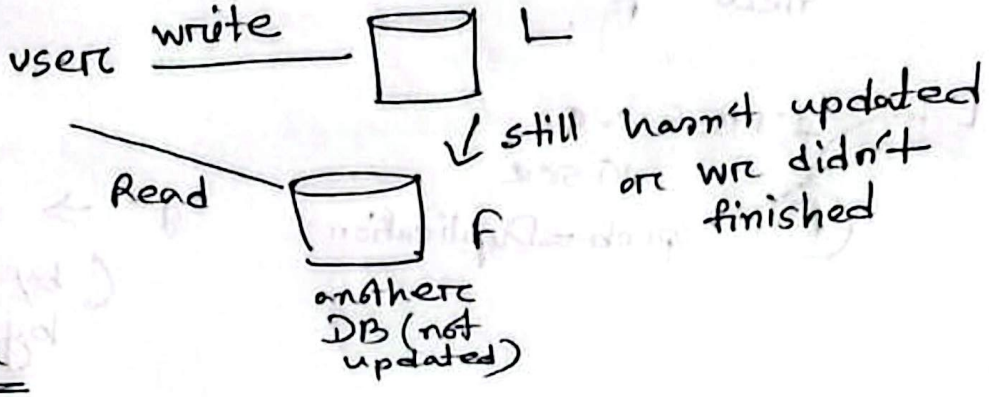
// how do we know about the crash?

⇒ In a specific time interval if the node doesn't sent any signal back

L to F data passing

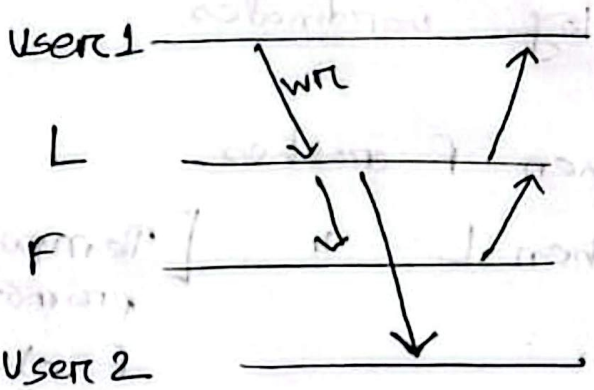
Replication lag — time to update

refresh — read req



Soln

Read Your Own writes



- query showing in phone LLM but not in pc

→ Soln — same user, same version

! — | user -f fixed

(2.) Multileader Replication

large app — lots of data centers
→ then → regionwise different leader
L to L data passing

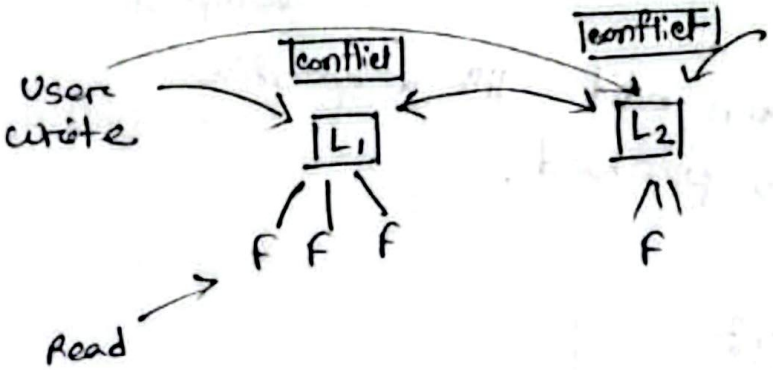
use cases
- a. Data Center
- b. Colab
- c. Clients with offline operation

(3.) Leaderless

Db Replication

1. Leader - follower
2. Multileader
3. Leaderless

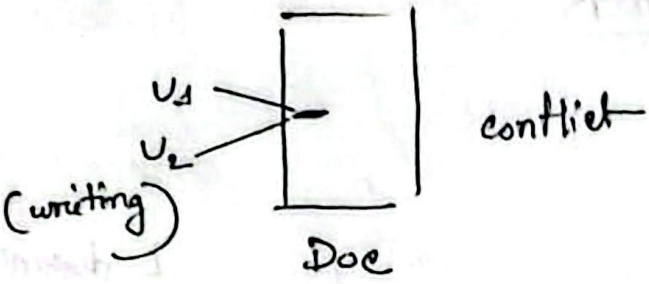
L doesn't have to be data centers. can be also mobiles, pc



→ same page
2 user editing and click saved at the same time in 2 different L.
Conflict

So before write conflict resolution happens.

(1) Data Center
(2) Clients with offline operation (tab, pc, phone all act as a individual L)



Soln → only one can write (not a good design)

another soln → can edit till a specific key is pressed.

avoid conflict
resolve conflict

only one L will have all the write (for a particular page) request, so this L will make other writes wait if one is already writing.

Which L to choose?

→ the first one that the page was written into.

same functionality but different user can write, one user will write in the same L it wrote before.

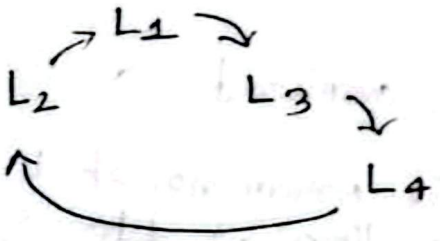
1. Conflict Avoid
2. " Converge (which data to show in case of conflict)

Conflict Solve in Write
- log in L1, log in L2; if as there's version diff. we will pass it to conflict resolve
- in Read
  user can choose which version to read.

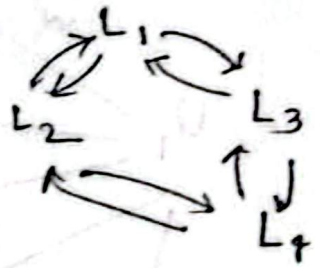
- there must be milisec, microsec difference in each write
  So, we will give unique id to all then accept the latest id (last)
- give each replica a unique id (doesn't matter write id. if someone writing a IUT page from IUT, it will be preffered than someone who's editing. from dhoke)
- merge
  show all user will choose which one is better
- Explicit DS

* converge & resolve works together

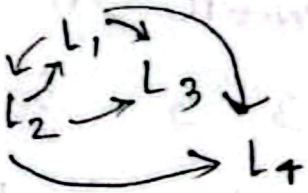
1st way
circular topology



2nd way
with neighbour, star topology

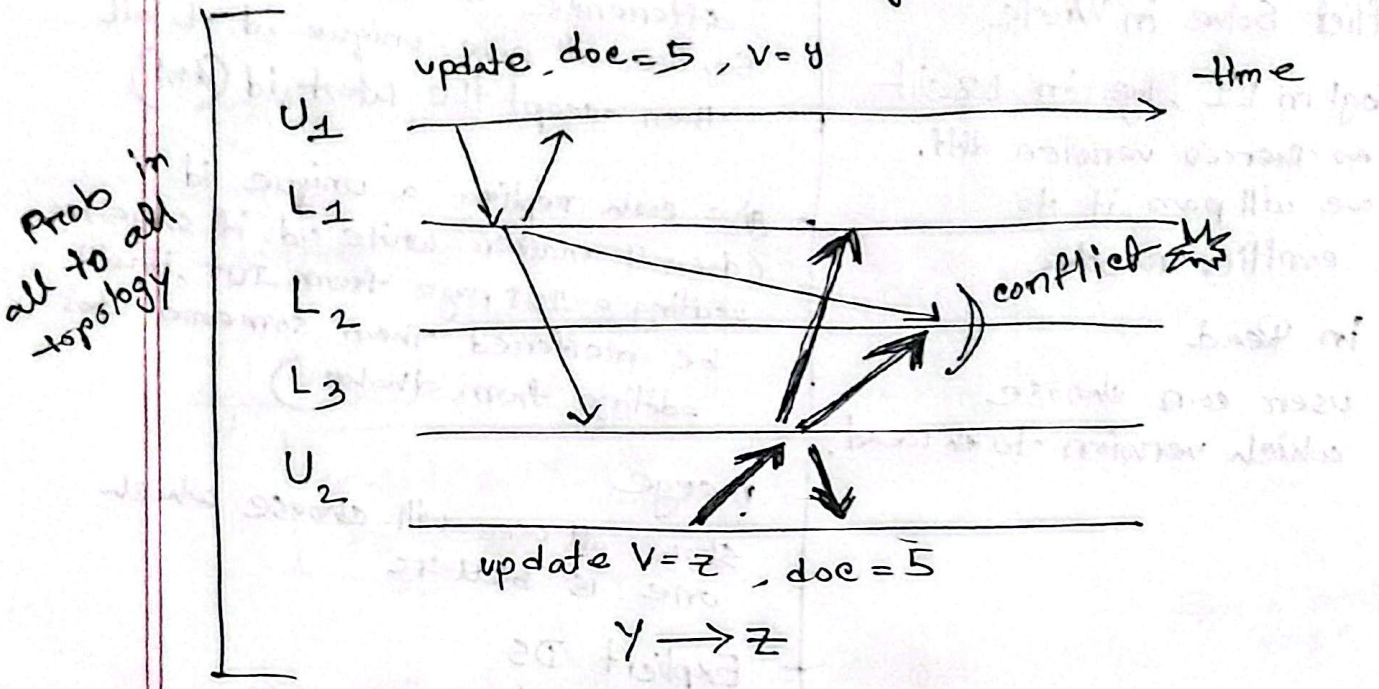


3rd way
all to all system



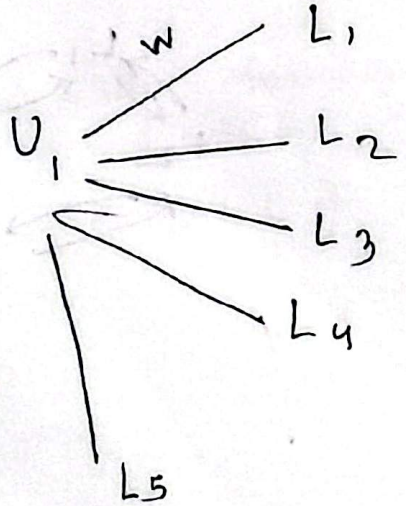
Problem → while updating in all L, one L may again change

Soln — notice versioning



← which L to use (multi or leaderless) upto the architect

Leaderless



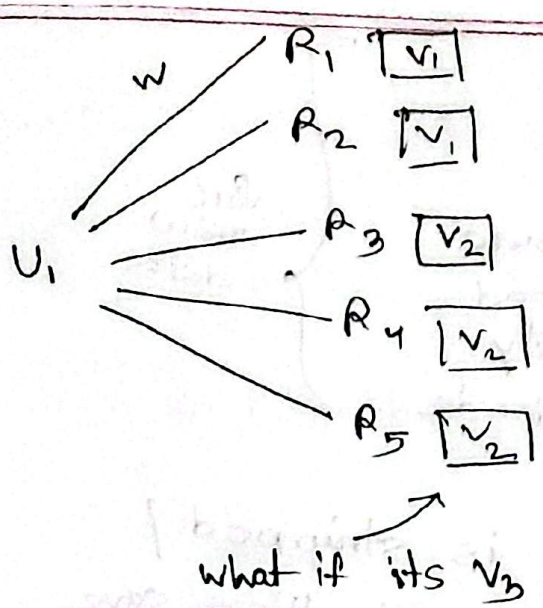
overhead

if certain no. of L, tells replicas got data successfully then successful (more than half)

$w + r > n$

3 3 5

L — here its replica everyone is L & F both



now which version to take ??

→ depends on architect

- most
- ~~latest~~ version replica (highest num)
- updated " "

→ then its become

W = 1 from 3

(Evolutionary Database Design)

EDD

Incrementally updating our db according to . . .

1. user Table (user id, name, age, gender) → 1 m entry already

want to update (add) location col^m

⟹ Alter table , on not null with default value

» now we can't add new data without location

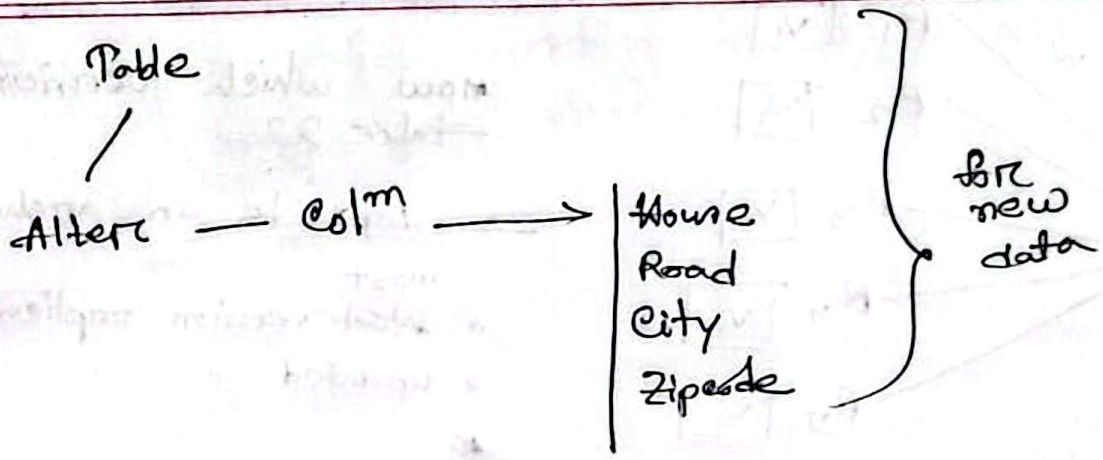
---

now, Table (—, —, —, —, location)

if we want to search by zipcode ?

⇒ cause old data don't have location, then what to do for that ⟹ ?

house
Road
City
zipcode



old data - ~~dest~~ String is stripped / streamed / out into small then save in altered colm, then dropped the prev one.

~~xxx~~ name

↓

full name

-> ? ~~first~~ copy then drop.

continuous Process

EDD

- Non Destructive (Do not have to modify the existing code or data)
- Destructive

as the sys is running we don't want to hamper user data (existing info)

Practices

1. DBAs collaborate with dev
2. Everything is version controlled
3. All changes are migration
4. Everybody has their own db
5. dev continuously integrate db changes
6. A db is of schema & Data
7. All changes are refactoring
   - → Db Schema
   - → Data Migration
   - → change in the access code

In a user table we want user_name instead of name (exists) so, we need destructive way.

- → Create that new col^m
- → Copy info's of prev. col^m
- (Do some testing then)
- → Now Delete prev col^m

Another exp → Inventory

Inv_code

Dhk_1230_01

we want =>

Dhk_1230_01

→ we can do string manipulation (like split when there's underscore) but what about 10M data

- so, add a new division col^m
- copy and Do string manipulation on that col^m
- Delete the prev

Change Log

if we make any kind of modification we must store that data.

change log table

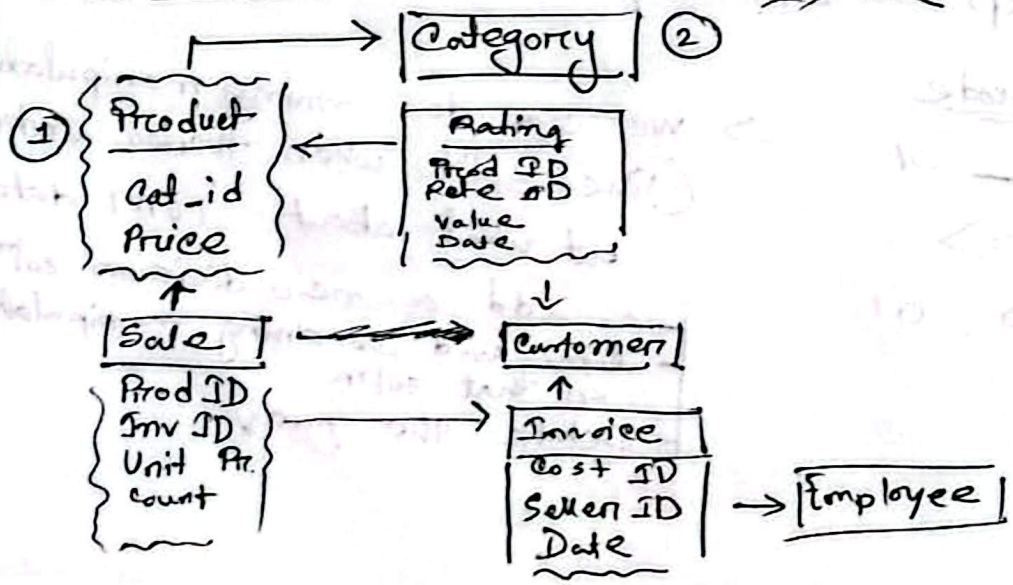
id	created at	created by	script name	description
			0000_chang.sql	

automatically increment script name after each modification (can be manual too for our lab)

This data (dev phase) doesn't go to production level data.
In dev phase we work on copied data

Reporting database

→ for



Read / Write

total sales of top 5 categories in May (read action is more than wr)

We need separate db from our transactional database.

Separate db used for reporting (uses lots of joins)

~~~

Operational Db
Reporting Db — Sync Prob (varies from company to company
- maybe in off peak hrs
- may be in every 10 min interval

Prob → might see old data while reporting

Video views → Comes from reporting db

if we want real time data — event based change (costly)

~~~

If we want to report from operation table / db we need to use lots of joins and it takes a lots of time.

So, we denormalize the db

We create a replication of operation db by copying it. Thats Reporting db..

Message Syne :

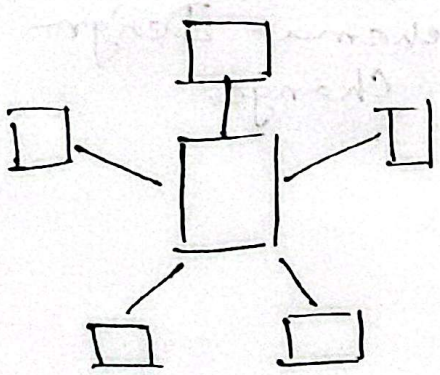
whenever theres changes in oper. db, we trigger an event, then the reporting db knows

↓

So, event driven

* Schedule shift

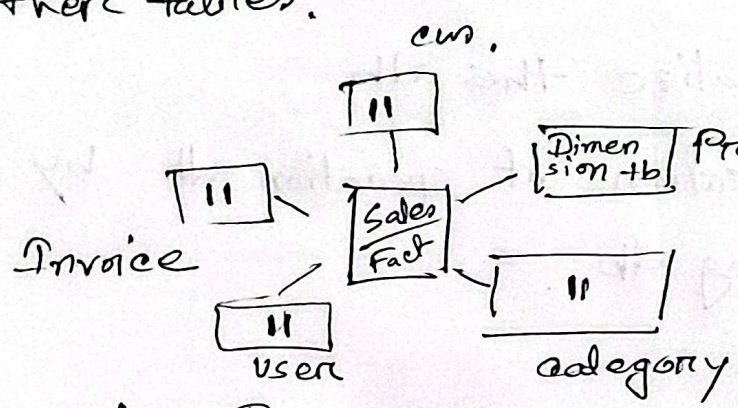
Star Schema (Simplest one)



we only keep the necessary infos in the reporting db. like in ecommerce we don't need the height data /kb

Fact Table → hold most informations

Master table that has connection with other tables.



Dimension Tb

other tb's that are connected to the fact table.

Instead of joining 5 tables we will keep redundant data. Will connect 2 tb with foreign key.

↓

Schema Design Change

In code

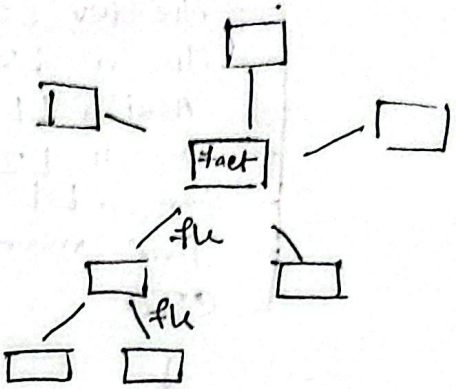
- open. db
- make instance (copy)
- then alter (create fact tb & connect other tb with fk)

OpDb	RpDb
~~Dim~~ Employee	fk (Dim-Employee
Sales	Fact_Sales

*** Dim-Dim relation ~~no~~ must not be none, otherwise it becomes open. tb / tb

Snowflakes Schema

denormalized version of star.

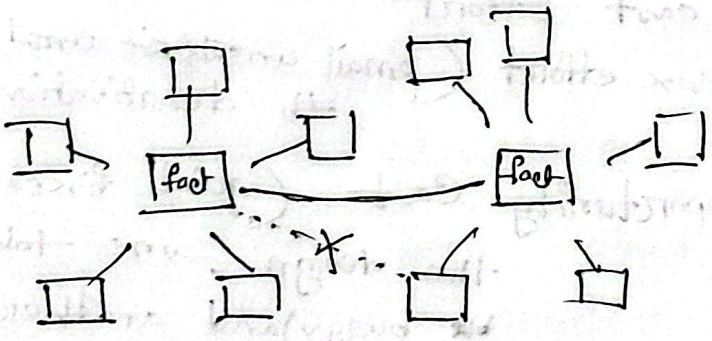


2 layers of data

Here instead of 5 layers join, we are having 2 layers join

Galaxy flake (for enterprises)

multiple fact tb exist. They are connected.



if multiple process are running, for each there's separate fact table.

Software Design

Solid
- SRP
- OCP
- LSP
- ISP
- DIP

OCP

Emp
- Jr dev L1
- Jr " L2
- Assi " L1
- " " L2
- Sen " L1
- line manager
- CTO

↓

we shouldn't use cond. here rather use polymorp. we use the instance of child class. (Inheritence)

Principles

1. Principle of least Astonishment (POLA)
   - ↓ Func name is indicating to something but func. doing diff thing.
   - ↘ logo click takes to reg instead of Home (POLA is mainly for UI)
2. Least Effort
3. Max effort (email ~~unsubscribe~~ subscribtion, ↓ deactivation)
4. Opportunity Cost (quick fixes or changing the design, one takes less time (but ~~the~~ buggy) and another is better but takes more time)
5. DRY
6. YAGNI
7. Keep it Simple